

PROYECTO “SERVICIOS TÉCNICOS”

Navegación, sesión y protección de rutas en React

React Router + estado de autenticación + organización de servicios API

1. Propósito

En esta etapa se continuará el proyecto Servicios Técnicos a partir del Login de la Etapa 3. El objetivo es que, después de autenticarse correctamente, el usuario pueda acceder a una pantalla principal y navegar entre diferentes vistas mediante React Router. También se organizará la comunicación con el backend para dejar preparada la aplicación para los siguientes módulos.

2. Objetivos de aprendizaje

- Instalar y utilizar React Router.
- Crear navegación entre páginas del frontend.
- Mantener un estado sencillo de autenticación.
- Redirigir al usuario después de un login correcto.
- Proteger una ruta para impedir el acceso directo al Dashboard sin autenticación.
- Implementar la función Cerrar sesión.
- Separar las llamadas a la API en una carpeta services.
- Comprender la diferencia entre autenticación, navegación y autorización.
- Registrar los cambios en Git y GitHub.

3. Requisitos previos

- Etapa 3 terminada.
- El backend Flask debe responder en http://127.0.0.1:5000.
- La API POST /api/login debe funcionar.
- El frontend React debe abrir en http://localhost:5173/.
- El Login debe permitir validar username y contraseña.
- Git y GitHub deben estar configurados.

4. Resultado esperado

```
Antes:
Login → API Flask → MySQL

Después de esta etapa:
Login
  | credenciales correctas
```




```
▼
Dashboard
  ├── Inicio
  ├── Servicios
  └── Cerrar sesión

Acceso directo a Dashboard sin sesión → Login
```

5. Verificar los dos servidores

Terminal 1 – Backend:

```
cd backend
venv\Scripts\activate
python -m app.main
```

Debe aparecer:

```
Running on http://127.0.0.1:5000
```

Terminal 2 – Frontend:

```
cd frontend
npm run dev
```

Debe aparecer:

```
Local: http://localhost:5173/
```

6. Instalar React Router

En la terminal del frontend ejecuta:

```
npm install react-router-dom
```

Espera a que npm termine. Después puedes comprobar que la dependencia aparezca en package.json.



7. Crear la estructura de carpetas

Dentro de frontend/src crea manualmente:

```
src/
├── components/
├── context/
│   └── AuthContext.jsx
├── pages/
│   ├── Login.jsx
│   └── Dashboard.jsx
├── services/
│   └── api.js
├── App.jsx
├── App.css
└── main.jsx
```

Las carpetas y archivos pueden crearse desde el explorador de VS Code con clic derecho → New Folder / New File.

8. Crear el servicio de API

Crea:

```
frontend/src/services/api.js
```

Coloca:

```
const API_URL = "http://127.0.0.1:5000";

export async function login(username, password) {
  const respuesta = await fetch(`${API_URL}/api/login`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      username,
      password,
    }),
  });

  const datos = await respuesta.json();

  return {
    ok: respuesta.ok,
```




```
    status: respuesta.status,
    datos,
  };
}
```

La ventaja de este archivo es que la URL del backend queda centralizada. En etapas posteriores se podrán agregar aquí las funciones para clientes, técnicos, servicios y otros módulos.

9. Crear AuthContext

Crea:

```
frontend/src/context/AuthContext.jsx
```

Coloca:

```
import { createContext, useContext, useState } from "react";

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [usuario, setUsuario] = useState(() => {
    const guardado = localStorage.getItem("usuario");
    return guardado ? JSON.parse(guardado) : null;
  });

  const iniciarSesion = (datosUsuario) => {
    setUsuario(datosUsuario);
    localStorage.setItem("usuario", JSON.stringify(datosUsuario));
  };

  const cerrarSesion = () => {
    setUsuario(null);
    localStorage.removeItem("usuario");
  };

  return (
    <AuthContext.Provider
      value={{ usuario, iniciarSesion, cerrarSesion }}
    >
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
```




```
  return useContext(AuthContext);
}
```

Nota didáctica: localStorage se utiliza aquí para conservar un indicador sencillo de sesión entre recargas. No se debe guardar una contraseña en localStorage. La autenticación real y el manejo seguro de tokens se mejorarán en una etapa posterior.

10. Configurar AuthProvider en main.jsx

Abre:

```
frontend/src/main.jsx
```

Utiliza una estructura como:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import { AuthProvider } from "./context/AuthContext.jsx";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </StrictMode>,
);
```

BrowserRouter permite utilizar rutas en React y AuthProvider pone el estado de sesión disponible para los componentes.



11. Crear la página Login.jsx

Crea:

```
frontend/src/pages/Login.jsx
```

Coloca:

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { login } from "../services/api";
import { useAuth } from "../context/AuthContext";
import "../App.css";

function Login() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [mensaje, setMensaje] = useState("");
  const [exito, setExito] = useState(false);
  const navigate = useNavigate();
  const { iniciarSesion } = useAuth();

  const manejarLogin = async (e) => {
    e.preventDefault();
    setMensaje("");

    if (!username || !password) {
      setMensaje("Username y contraseña son obligatorios");
      setExito(false);
      return;
    }

    try {
      const resultado = await login(username, password);

      if (resultado.ok && resultado.datos.success) {
        iniciarSesion({ username: resultado.datos.username });
        setExito(true);
        setMensaje(resultado.datos.message);
        navigate("/dashboard");
      } else {
        setExito(false);
        setMensaje(resultado.datos.message || "Usuario o contraseña incorrectos");
      }
    } catch (error) {
      setExito(false);
```




```
        setMensaje("No se pudo conectar con el servidor");
      }
    };

    return (
      <div className="contenedor">
        <div className="login">
          <h1>Servicios Técnicos</h1>
          <h2>Iniciar sesión</h2>
          <form onSubmit={manejarLogin}>
            <label>Username</label>
            <input
              type="text"
              value={username}
              onChange={(e) => setUsername(e.target.value)}
              placeholder="Ingrese su username"
            />
            <label>Contraseña</label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              placeholder="Ingrese su contraseña"
            />
            <button type="submit">Ingresar</button>
          </form>
          {mensaje && (
            <p className={exito ? "mensaje exito" : "mensaje error"}>
              {mensaje}
            </p>
          )}
        </div>
      </div>
    );
}

export default Login;
```



12. Crear la página Dashboard.jsx

Crea:

```
frontend/src/pages/Dashboard.jsx
```

Coloca:

```
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import "../App.css";

function Dashboard() {
  const { usuario, cerrarSesion } = useAuth();
  const navigate = useNavigate();

  const salir = () => {
    cerrarSesion();
    navigate("/");
  };

  return (
    <div className="dashboard">
      <h1>Servicios Técnicos</h1>
      <h2>Panel principal</h2>
      <p>Bienvenido, <strong>{usuario?.username}</strong></p>

      <nav>
        <Link to="/dashboard">Inicio</Link>
      </nav>

      <button onClick={salir}>Cerrar sesión</button>
    </div>
  );
}

export default Dashboard;
```



13. Crear una ruta protegida

Crea:

```
frontend/src/components/RutaProtegida.jsx
```

Coloca:

```
import { Navigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

function RutaProtegida({ children }) {
  const { usuario } = useAuth();

  if (!usuario) {
    return <Navigate to="/" replace />;
  }

  return children;
}

export default RutaProtegida;
```

La ruta protegida comprueba si existe un usuario en el contexto. Si no existe, redirige al Login.

14. Configurar App.jsx

Abre:

```
frontend/src/App.jsx
```

Reemplaza su contenido por:

```
import { Routes, Route, Navigate } from "react-router-dom";
import Login from "./pages/Login";
import Dashboard from "./pages/Dashboard";
import RutaProtegida from "./components/RutaProtegida";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Login />} />
      <Route
        path="/dashboard"
        element={
          <RutaProtegida>
            <Dashboard />
```




```
          </RutaProtegida>
        }
      />
      <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
  );
}

export default App;
```

15. Agregar estilos para Dashboard

Al final de frontend/src/App.css agrega:

```
.dashboard {
  min-height: 100vh;
  padding: 40px;
  text-align: center;
}

.dashboard nav {
  margin: 25px 0;
}

.dashboard nav a {
  margin: 0 10px;
  text-decoration: none;
}

.dashboard button {
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
```

16. Guardar y verificar errores

Guarda todos los archivos con Ctrl + S. Observa la terminal de Vite y el panel Problems de VS Code.

Si Vite muestra un error de importación, revisa:

- Que el nombre de la carpeta sea exactamente pages, services, context o components.
- Que los nombres de archivo coincidan con los imports.
- Que las rutas relativas comiencen con ../ cuando corresponda.
- Que react-router-dom esté instalado.



17. Prueba 1: abrir Login

1. Abre http://localhost:5173/.
2. Comprueba que aparezca el Login.
3. No debe aparecer todavía el Dashboard.

Resultado esperado: Login visible.

18. Prueba 2: credenciales correctas

4. Escribe un usuario válido de tecnico.usuario.
5. Escribe la contraseña correspondiente.
6. Presiona Ingresar.

Resultado esperado:

```
1. React envía POST /api/login.
2. Flask consulta MySQL.
3. Flask devuelve success=true.
4. React guarda el usuario en AuthContext.
5. React navega a /dashboard.
6. Dashboard muestra el username.
```

19. Prueba 3: credenciales incorrectas

Utiliza una contraseña incorrecta.

Resultado esperado:

```
Usuario o contraseña incorrectos
```

El usuario debe permanecer en el Login y no debe acceder al Dashboard.

20. Prueba 4: acceso directo al Dashboard

En el navegador escribe directamente:

```
http://localhost:5173/dashboard
```

Si no existe una sesión, RutaProtegida debe redirigir automáticamente al Login.



21. Prueba 5: cerrar sesión

7. Inicia sesión correctamente.
8. Comprueba que aparezca Dashboard.
9. Presiona Cerrar sesión.

Resultado esperado:

```
Se elimina el usuario del estado y de localStorage y se regresa al Login.
```

10. Intenta entrar nuevamente a /dashboard.

Debe regresar al Login.

22. Prueba 6: recargar el navegador

11. Inicia sesión.
12. En Dashboard presiona F5.

El usuario debe conservarse porque esta etapa guarda un indicador de sesión en localStorage. Esto es únicamente una solución didáctica; no se deben guardar contraseñas allí.

23. Conceptos importantes

Concepto	Función
React Router	Permite navegar entre vistas sin recargar toda la aplicación.
BrowserRouter	Habilita el enrutamiento basado en la URL.
Routes / Route	Definen las rutas de la aplicación.
Navigate	Permite redireccionar.
useNavigate	Permite cambiar de ruta desde código.
AuthContext	Centraliza el estado de autenticación.
localStorage	Conserva datos simples en el navegador.
RutaProtegida	Impide mostrar una vista si no hay sesión.
services/api.js	Centraliza las llamadas al backend.



24. Arquitectura resultante

```
frontend
│
├── pages
│   ├── Login.jsx
│   └── Dashboard.jsx
│
├── components
│   └── RutaProtegida.jsx
│
├── context
│   └── AuthContext.jsx
│
├── services
│   └── api.js
│
└── App.jsx
        │
        ▼
    Flask
        │
        ▼
    MySQL
```

25. Buenas prácticas

- No colocar contraseñas de MySQL dentro de React.
- No guardar contraseñas del usuario en localStorage.
- Mantener las URL y llamadas HTTP organizadas en services.
- Separar páginas, componentes y contexto.
- Utilizar rutas protegidas para las pantallas que requieren sesión.
- Usar Git para registrar cambios pequeños y descriptivos.
- Captura de la terminal Flask mostrando POST /api/login.
- Captura del repositorio GitHub actualizado.



26. Tabla de pruebas

Prueba	Acción	Resultado esperado	Resultado	Evidencia
Login correcto	Credenciales válidas	Dashboard		
Login incorrecto	Contraseña incorrecta	Permanece en Login		
Dashboard directo	Abrir /dashboard sin sesión	Redirección a Login		
Cerrar sesión	Presionar botón	Regresa a Login		
Recarga	F5 después de login	Sesión didáctica conservada		

27. Preguntas de análisis

13. ¿Qué problema resuelve React Router?

Respuesta: _____

_____

14. ¿Qué diferencia existe entre una página y una ruta en React?

Respuesta: _____

_____

15. ¿Qué función cumple AuthContext?

Respuesta: _____

_____

16. ¿Por qué se utiliza una ruta protegida?

Respuesta: _____

_____

17. ¿Qué ocurre cuando el usuario intenta entrar directamente a /dashboard?

Respuesta: _____

_____



18. ¿Por qué no debemos guardar la contraseña en localStorage?

Respuesta: _____

_____

19. ¿Qué ventaja tiene separar login en services/api.js?

Respuesta: _____

_____

20. ¿Qué diferencia existe entre autenticación y autorización?

Respuesta: _____

_____

21. ¿Qué papel continúa desempeñando Flask?

Respuesta: _____

_____

22. ¿Qué papel continúa desempeñando MySQL?

Respuesta: _____

_____

28. Git y GitHub

Desde la raíz del proyecto ejecuta:

```
git status
git add .
git commit -m "Implementa navegación y protección de rutas"
git push
```

Verifica que el commit aparezca en GitHub.